Отчёт по Д31, шаг 2: охота на баги

Вариант seed 42 · три семантические ошибки · обучение и генерация

Кратко. В ноутбуке найдены и устранены три ошибки, которые не мешали коду запускаться, но меняли смысл вычислений. До исправлений проходило 7 из 17 проверок, после — 17 из 17. Исправленная модель обучена 600 шагов на игрушечном корпусе: потери упали с 3.186 до 0.035, валидационные — с 3.064 до 0.018. Генерация воспроизводит структуру корпуса, а не случайные символы.

1. Найденные ошибки

Все три ошибки — семантические: интерпретатор их не замечает, обучение идёт, метрики выглядят правдоподобно, но вычисляется не то, что заявлено в docstring. Ниже — локализация, механизм нарушения и исправление.

№	Локализация	Механизм нарушения и причина сбоя	Как исправлено
1	LayerNorm.forward, строки 60–61 (ячейка c023)	Среднее и дисперсия считаются по dim=0 (батч), а не по dim=-1 (признаки внутри токена). Для входа (B, L, D) это превращает LayerNorm в batch-нормировку: статистики каждого канала усредняются по всем примерам и позициям батча, поэтому токен нормируется не сам по себе, а относительно соседей. При генерации B=1 среднее по единственному элементу равно самому x, дисперсия равна нулю, и (x − mean) / sqrt(0 + eps) = 0: слой возвращает только bias. Вход перестаёт влиять на выход; при нулевом bias логиты всех позиций одинаковы, softmax равномерен, и top-k выбирает токен случайно. На обучении результат зависит от состава батча — возникает рассогласование train/inference.	dim=0 → dim=-1 в x.mean(...) и x.var(...): статистики считаются по D признакам каждого токена независимо, как и требует LayerNorm.
2	scaled_dot_product_attention, строка 101 (ячейка c023)	Softmax берётся по dim=-2 (ось запросов) вместо dim=-1 (ось ключей). Матрица scores имеет форму (B, H, L_q, L_k); при нормировке по запросам каждая строка-запрос перестаёт быть распределением по ключам: сумма весов по ключам не равна единице, выход внимания — не выпуклая комбинация value, а его масштаб зависит от позиции. Вес ключа j нормируется по всем запросам, включая будущие i > текущего, поэтому изменение будущего токена меняет веса в прошлых позициях: причинная маска не спасает, информация течёт из будущего. На обучении модель использует этот «подсмотр» (teacher forcing), а при авторегрессионной генерации будущих токенов нет — распределение входов резко отличается, и генерация рассыпается.	dim=-2 → dim=-1: softmax по последней оси (ключам). Каждая строка матрицы внимания — распределение с суммой 1, будущие позиции получают ровно нулевой вес.
3	sinusoidal_positional_encoding, строка 39 (ячейка c023)	Позиции начинаются с 1: arange(1, length + 1) вместо arange(0, length). Токен с индексом p получает фазу (p+1)ω: первый токен кодируется как вторая позиция, а последний — фазой length, выходящей за спроектированный диапазон 0..length−1. Пропадает нулевой якорь (sin 0 = 0, cos 0 = 1), абсолютная позиционная разметка систематически смещена на единицу. Относительные расстояния при глобальном сдвиге сохраняются, поэтому эффект мягче двух других ошибок, но контракт функции нарушен: модель обучается на неверных абсолютных позициях, а проверки фиксируют расхождение с формулой.	arange(1, length + 1) → arange(0, length): позиция 0 даёт sin = 0, cos = 1, как в формуле «Attention is All You Need».

2. Результаты проверок

Проверки: 7/17 → 17/17. Исходный код проходил 7 из 17. Падали 2 проверки позиционного кодирования, 2 — LayerNorm, 4 — внимания (эталон, сумма весов, масштаб, маска) и 2 — генерации (top-k = 1 и режим eval). После правок все 17 зелёные, включая обучение на крошечной последовательности. Проверки открыты в ноутбуке и прогоняются ровно те же, что и при приёме.

Показательно, что проверка «модель не подглядывает в будущее» на исходном коде формально проходила: ошибка LayerNorm при B=1 обнуляла логиты и маскировала утечку. После её исправления проверка стала бы падать, поэтому обе ошибки нужно исправлять вместе.

3. Обучение и валидация

Короткий цикл на игрушечном корпусе: одна и та же фраза повторена 200 раз. Данные нарочно простые — задача не обучить LLM, а показать, что после исправлений модель действительно учится и генерирует структурированный текст. Хвостовые 10% корпуса отданы под валидацию и не участвуют в обновлении весов.

Параметр	Значение
Корпус	«алгоритм внимания смотрит на все позиции сразу. » × 200; 9600 символов, алфавит 19 символов
Контекст / батч	64 токена / 32
Модель	d_model = 128, 4 головы, 2 слоя, dropout = 0.1
Оптимизатор	AdamW, lr = $3 \cdot 10^{-4}$
Обучение	600 шагов; 90% train / 10% validation
Итог	train loss 3.186 → 0.035; validation loss 3.064 → 0.018

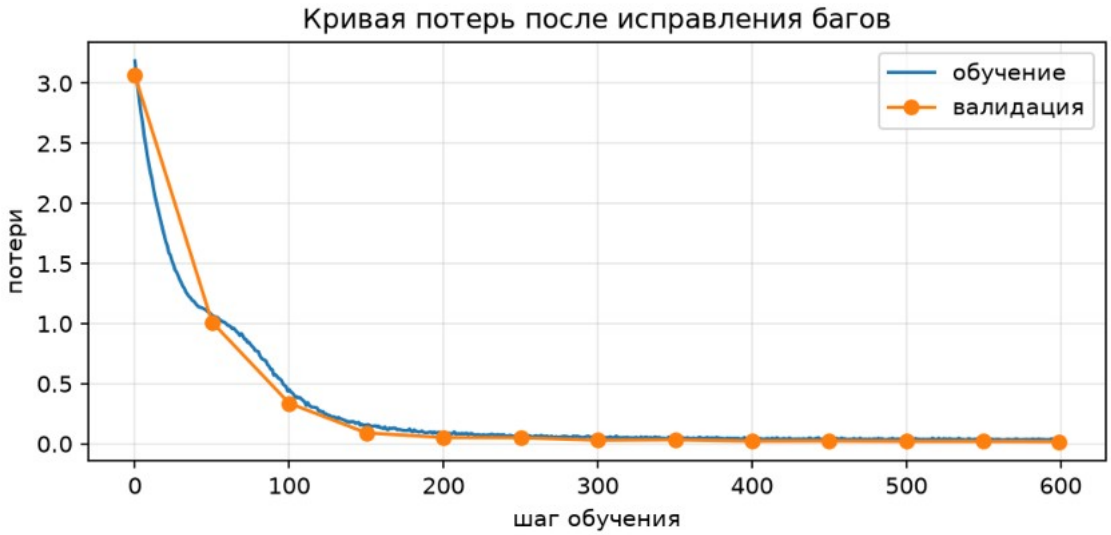


Рис. 1. Кривая потерь после исправления ошибок: обучение и валидация.

4. Пример генерации

Параметры: стартовый токен «а», max_new_tokens = 200, temperature = 0.8, top_k = 10. Модель продолжает с середины слова («азу.»), затем точно воспроизводит повторяющийся шаблон — это и есть ожидаемый результат для игрушечного корпуса.

```
азу. алгоритм внимания смотрит на все позиции сразу. алгоритм внимания смотрит на все позиции сразу.
алгоритм внимания смотрит на все позиции сразу. алгоритм внимания смотрит на все позиции сразу. алго
```

5. Вывод

Все три ошибки меняли статистики нормализации, распределение внимания и абсолютные позиции. После исправления проверки зелёные, потери падают, а генерация воспроизводит структуру корпуса — причина была в семантике вычислений, а не в архитектуре или данных.